

Shubhangi Jeve

B.Tech in Artificial Intelligence & Data Science
Terna College of Engineering, Dharashiv | CGPA: 8.5
Hyderabad, Telangana

+91-9579122372
shubhangijeve@gmail.com
github.com/ShubhangiJeve
linkedin.com/in/shubhangi-jeve-97445b235

CAREER OBJECTIVE

AI Engineer with hands-on production experience building end-to-end LLM-powered systems, RAG pipelines, and full-stack AI applications. Proficient in Python, FastAPI, vector databases, and LLM integration using Groq, Gemini, and Hugging Face. Seeking an AI Engineer role to architect and deploy scalable, enterprise-grade intelligent systems.

EDUCATION

• **Bachelor of Technology in Artificial Intelligence and Data Science** 2021 – 2025
Terna Public Charitable Trust's College of Engineering, Dharashiv CGPA: 8.5

TECHNICAL SKILLS

Languages & Frameworks: Python, SQL, FastAPI, Flask, Next.js 14 (TypeScript), Streamlit
AI and Machine Learning: Regression, Classification, Clustering, Feature Engineering, ANN, CNN, Fine-tuning, Prompt Engineering
Generative AI and LLMs: Retrieval-Augmented Generation (RAG), LangChain, LangGraph, Groq API, Google Gemini, Ollama, Open-source LLMs
Embeddings and Vector Search: sentence-transformers (BGE-M3, multilingual-e5), FAISS, ChromaDB, pgvector, Hybrid Semantic and Lexical Search
Libraries: NumPy, Pandas, Scikit-learn, TensorFlow/Keras, PyTorch, NLTK, spaCy, Hugging Face Transformers, Pydantic v2, SQLAlchemy 2.0
Databases: PostgreSQL with pgvector, MySQL, ChromaDB, FAISS Vector Store
DevOps and Tools: Docker, Docker Compose, Git, GitHub, VS Code, Jupyter Notebook, N8N, Observability Tools

EXPERIENCE

• **AI Engineer Intern** January 2026 – Present
COGNITBOTZ Hyderabad, Telangana

- **Project: LegalAID** — Enterprise-grade, AI-powered Legal Research and Litigation Assistance Platform for Indian court case data, combining Retrieval-Augmented Generation (RAG), semantic search, and LLM-driven drafting into a production-ready system.
- **RAG Pipeline Architecture:** Designed and implemented a multi-stage hybrid retrieval pipeline combining dense vector search (pgvector with BAAI/bge-m3 embeddings) and PostgreSQL full-text search for lexical recall. Applied parent-child chunk retrieval with entity-aware chunking strategies tailored per legal document type including case summaries, hearing history, orders, and procedural timelines to maximize citation-grounded answer quality.
- **LLM Integration and Hallucination Control:** Integrated Groq-hosted LLMs through a provider-abstracted service layer supporting model swapping at configuration time. Enforced a strict abstention policy where the system returns “Insufficient evidence” when retrieval confidence falls below a defined threshold. Validated every generated answer against cited source chunks to minimize hallucination in legal responses.
- **Data Ingestion and Normalization Engine:** Built a robust ingestion pipeline that parses yearly Indian court JSON dumps containing 20,000 to 75,000 cases spanning 25 years. The pipeline normalizes 30+ canonical fields including parties, acts, bench type, coram, and hearing history, deduplicates records, generates semantic embeddings, and writes structured data with vector indexes in a single resumable run.
- **Backend API Development (FastAPI and SQLAlchemy 2.0):** Developed a fully asynchronous REST API with complete Pydantic v2 schema validation, exposing endpoints for case search, RAG-based question answering, AI-assisted legal document drafting (notices, replies, case briefs, legal memos), litigation comparison, and a dashboard analytics module.
- **Frontend Development (Next.js 14 and TypeScript):** Built the complete frontend in Next.js 14 with TypeScript, covering a natural-language search workspace, faceted case explorer, structured case detail pages with hearing timelines, a legal drafting workspace, side-by-side case comparison view, and an admin console for ingestion pipeline management.
- **Vector Database and Hybrid Search:** Configured PostgreSQL with pgvector as the unified store for both structured metadata queries and approximate nearest-neighbor (ANN) vector retrieval, achieving sub-4-second search latency on the full case corpus while eliminating the overhead of a separate vector database.
- **Containerized Deployment with Docker:** Packaged the entire system using Docker and Docker Compose, with the FastAPI backend running in a lightweight Python 3.11 image and the vector-enabled database on PostgreSQL 16 with pgvector. Implemented health-check dependency chains ensuring correct startup order and persistent named volumes for data durability.

- **Project: MeetOps** (*May 2026 – Present*) — Enterprise-grade AI-powered Meeting Copilot and Project Intelligence Platform integrating Microsoft Teams via ephemeral Dockerized Playwright bots, a RAG-driven knowledge engine, and an agentic LLM layer that transforms live meeting context into structured, actionable project intelligence.
- **RAG Pipeline and Vector Intelligence:** Designed a semantic RAG pipeline over meeting transcripts, BRDs, and project artifacts using OpenAI embeddings stored in pgvector. Applied speaker-turn and topical-boundary chunking (over fixed character splits) to preserve semantic coherence, significantly improving top-K retrieval precision for time-sensitive project queries.
- **Hallucination Control and Confidence Gating:** Enforced a strict abstention guardrail where the LLM is instructed to respond “Insufficient meeting context” when cosine similarity of retrieved chunks falls below 0.75, preventing fabricated technical details in high-stakes project conversations.
- **Agentic Tool-Calling and LLM Orchestration:** Implemented an agentic workflow using OpenAI function-calling to trigger structured actions mid-conversation, including Jira ticket creation, rolling meeting summaries, and live SAP GRC metrics retrieval, transforming the copilot from reactive Q&A into an autonomous project action layer.
- **Ephemeral Bot Orchestration (Docker + Playwright):** Engineered a Disposable Workspace bot pattern where per-meeting Playwright instances are dynamically provisioned and reaped via Docker with TTL watchdogs and health checks, ensuring zero state contamination across client sessions and efficient compute utilization.
- **Low-Latency Streaming and Caching:** Implemented a multi-tier Redis caching layer that short-circuits the full embedding and LLM generation cycle on repeated or semantically identical queries, and streamed LLM output directly to the React frontend via Server-Sent Events (SSE) to minimize perceived latency during live meetings.

•Artificial Intelligence Intern

February 2025 – April 2025

Infosys Springboard

Remote

- Built a **Healthcare RAG Chatbot** on a Wikipedia dataset using FAISS vector search, a Flask backend, and a React with Tailwind CSS frontend, with secure API endpoints designed for medically grounded responses.
- Implemented a real-time query processing pipeline with source verification and an interactive UI supporting multi-turn question answering sessions.

•Data Science Intern

January 2025 – June 2025

Adhyayan IT

Remote

- Built a **Text-to-SQL application** using LangChain and LLMs (Google Gemini, Groq) that converts natural language queries to executable SQL, with RAGAS evaluation achieving 100% context precision and high helpfulness scores.
- Developed an interactive Streamlit UI with secure MySQL connectivity, enabling non-technical users to query databases efficiently with reliable, context-aware responses.

•Data Analyst Intern

October 2023 – January 2024

Cyber Police Station

Dharashiv, Maharashtra

- Managed 500+ user credentials for the National Cyber Crime Reporting Portal and organized 1,000+ complaint records, improving case resolution efficiency by 30% and accelerating case progression by 20%.
- Communicated with victims to provide timely status updates, enhancing overall satisfaction by 15%.

PERSONAL PROJECTS

•RAG Document QA System

Production-grade AI-powered question answering on PDF and TXT documents using FastAPI and React

- Engineered a full-stack RAG system with a FastAPI backend and a React (TypeScript, Tailwind CSS) frontend, integrating document chunking, sentence-transformers embeddings, and ChromaDB vector search with source verification for accurate and traceable answers.
- Built asynchronous RESTful APIs and deployed the complete stack using Docker, with an interactive dashboard for monitoring ingestion KPIs and retrieval quality metrics in real time.

•Insurance Claim Automation System

AI-powered insurance claim processing using Google Gemini AI, LangChain, and Flask

- Developed an AI-powered insurance claim automation system using Google Gemini AI to process and validate medical claims from PDF documents, extracting key medical and billing information using natural language processing.
- Built a Flask web application with LangChain integration for intelligent claim assessment and decision-making, with robust error handling and data validation pipelines to ensure accurate processing and reduce manual review time.